DASS Assignment 2: Phase 1 White-Box Testing Report
MoneyPoly System

Student Roll Number: 2024101116

March 23, 2026

Contents

1 Objective **2**

2 Scope and Target Code **2**

3 1.1 Control Flow Graph **2**
- 3.1 How the CFG was built . . . 2
- 3.2 CFG attachment . . . 2
- 3.3 Node mapping used in the diagram . . . 3
- 3.4 Coverage expectation from CFG . . . 4

4 1.2 Code Quality Analysis with pylint **4**
- 4.1 Command used . . . 4
- 4.2 Baseline outcome . . . 4
- 4.3 Key warnings observed . . . 4
- 4.4 Iterative fix log format (required commits) . . . 5

5 1.3 White-Box Test Cases **5**
- 5.1 Test design strategy . . . 5
- 5.2 Proposed branch and state test set . . . 5
- 5.3 All Bugs and Fixes (Phase 1.3) . . . 6
- 5.4 Errors or logical issues found by white-box reasoning . . . 7
- 5.5 Required error-fix commit format . . . 7

6 Evidence Summary **8**

1 Objective

This report documents the white-box testing activity for the MoneyPoly implementation. The focus is on:

1. Control-flow understanding using a Control Flow Graph (CFG)
2. Code quality analysis using `pylint`
3. Branch-oriented white-box test design with edge-case coverage

2 Scope and Target Code

White-box analysis was performed on the MoneyPoly module files under:

- `2024101116/whitebox/code/moneypoly/`
- Existing tests under `2024101116/whitebox/tests/`

The core control-flow focus is the game turn lifecycle, especially:

- `Game.play_turn`
- `Game._move_and_resolve`
- `Game._handle_jail_turn`
- `Game._apply_card`

3 1.1 Control Flow Graph

3.1 How the CFG was built

A hand-drawn CFG was created for the main game-turn logic. Nodes represent either single statements or logically grouped statements. Directed arrows indicate control transitions for all decision outcomes.

3.2 CFG attachment

The hand-drawn CFG image is attached in the report in below Figure 1.

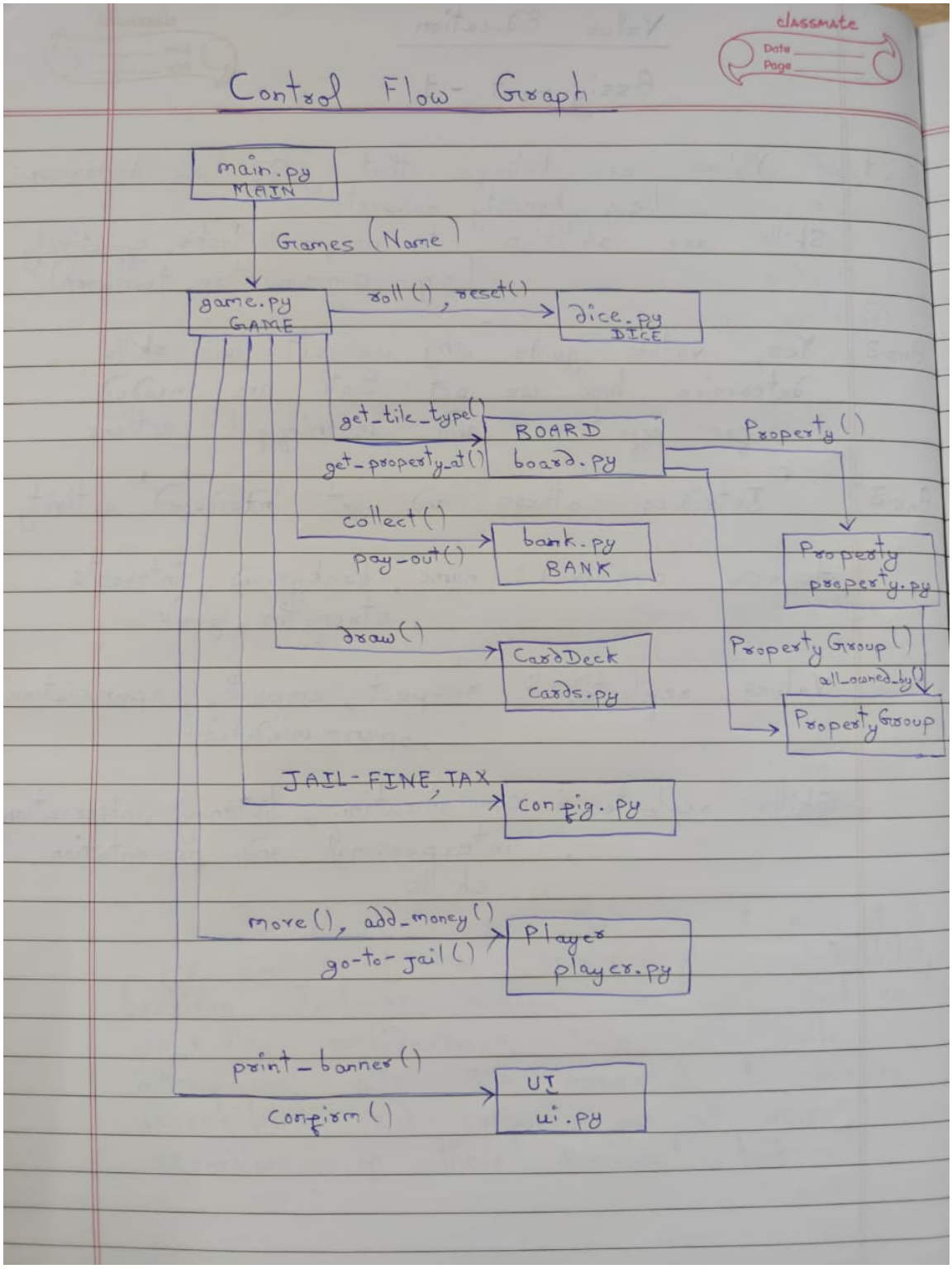

Figure 1: Hand-drawn Control Flow Graph for core MoneyPoly turn flow

3.3 Node mapping used in the diagram

The following node mapping is taken directly from the hand-drawn image in `diagrams/image.png`:

Node	Description

Node 1	`main.py` (entry point)
Node 2	`game.py` / `Game` class (central controller node)
Node 3	`dice.py` / `Dice`: called from game via `roll()`, `reset()`
Node 4	`board.py` / `Board`: called via `get_tile_type()`, `get_property_at()`
Node 5	`property.py` / `Property`: board/game interaction through property objects
Node 6	`property.py` / `PropertyGroup`: relation shown via `PropertyGroup()` and `all_owned_by()`
Node 7	`bank.py` / `Bank`: called via `collect()` and `pay_out()`
Node 8	`cards.py` / `CardDeck`: called via `draw()`
Node 9	`config.py`: constants flow into game (example in image: jail fine/tax values)
Node 10	`player.py` / `Player`: called via `move()`, `add_money()`, `go_to_jail()`
Node 11	`ui.py` / UI helpers: called via `print_banner()` and `confirm()`

3.4 Coverage expectation from CFG

The CFG indicates that complete branch coverage requires tests for:

- both outcomes of jail checks, doubles checks, and bankruptcy checks
- each tile-category branch in `_move_and_resolve`
- each action branch in card handling (collect, pay, jail, jail_free, move_to, birthday/collect_from_all)

4 1.2 Code Quality Analysis with pylint

4.1 Command used

```
python -m pylint code/moneypoly tests
```

4.2 Baseline outcome

Initial pylint score observed: **9.27/10**.
Despite a high score, multiple maintainability and style issues were reported.

4.3 Key warnings observed

- `superfluous-parens` in game menu index validation
- `missing-final-newline` in selected source files
- `attribute-defined-outside-init` in dice state setup
- `trailing-whitespace` in source/test files
- `missing-module-docstring`, `unused-import`, import-order warnings in tests
- design warnings such as `too-many-branches`, `too-many-instance-attributes`

4.4 Iterative fix log format (required commits)

Use one commit per iteration in the required format:

- `Iteration 1: Fix test import order, docstrings, and unused imports`
- `Iteration 2: Remove trailing whitespace and final newline warnings`
- `Iteration 3: Resolve dice init warning and simplify conditionals`

Note: Design warnings like `too-many-branches` may be acceptable in game-loop orchestration if behavior is clear and tested. If kept, justify in report.

5 1.3 White-Box Test Cases

5.1 Test design strategy

Tests were designed from internal control flow and data-state transitions, not only public behavior. The goal was to cover:

- all major branches
- critical variable state changes (balance, jail flags, ownership)
- edge cases (zero, negative, exact boundary amounts)

5.2 Proposed branch and state test set

Test ID	Why this test is needed	Expected check / issue targeted
WB-T1: Buy property exact balance	Verifies purchase boundary where player has exactly property price	Purchase succeeds, player balance becomes 0, ownership/property list updated
WB-T2: Buy property insufficient funds	Covers false branch in purchase logic	Purchase fails and no ownership/balance corruption
WB-T3: Rent payment transfer	Validates money transfer path for non-owner rent events	Tenant decreases, owner increases by same rent value
WB-T4: Rent on mortgaged property	Branch where rent must not be charged	No balance changes for either player
WB-T5: Trade success path	Verifies normal trade control flow and state transfer	Cash moves buyer→ $sellerandownershipchanges$
WB-T6: Trade negative cash edge case	Validates defensive branch for invalid input	Trade must fail; prevents illegal money flow or runtime errors

WB-T7: Jail voluntary fine path	Covers jail release option and state reset	Fine deducted from player, collected by bank, jail flags reset
WB-T8: Mandatory release after 3 turns	Verifies forced branch in jail handling	On 3rd turn, fine applied and player released
WB-T9: Bankruptcy elimination	Ensures elimination logic releases properties and removes player	Player marked eliminated, properties reset to unowned, removed from active list
WB-T10: Winner selection	Protects correctness of endgame decision logic	Richest net-worth player is returned

5.3 All Bugs and Fixes (Phase 1.3)

The following ten logical bugs were identified and fixed in the original MoneyPoly codebase:

Bug 1: Incorrect Dice Range
File: `whitebox/code/board.py`
Error: `randint(1, 5)` was used for a standard 6-sided die.
Fix: Changed to `randint(1, 6)`.

Bug 2: Net Worth Calculation Ignores Property Values
File: `whitebox/code/player.py`
Error: `net_worth()` returned only cash and ignored owned property values.
Fix: Added the sum of owned property values to the cash total.

Bug 3: GO Salary Awarded Only on Exact Landing
File: `whitebox/code/player.py`
Error: `move()` gave GO salary only when landing exactly on GO, not when passing GO.
Fix: Updated logic to award salary whenever the new position wraps around the board.

Bug 4: Property Purchase Balance Check Uses `<=` instead of `<`
File: `whitebox/code/player.py`
Error: Purchase was blocked even when player cash exactly equaled property price.
Fix: Changed condition from `<=` to `<`.

Bug 5: Rent Payment Does Not Transfer to Owner
File: `whitebox/code/player.py`
Error: Rent was deducted from tenant but not credited to owner.
Fix: Added owner cash increment by the rent amount.

Bug 6: Winner Determination Returns Loser
File: `whitebox/code/game.py`
Error: `find_winner()` used `min()` on net worth values.
Fix: Changed `min()` to `max()`.

Bug 7: Monopoly Check Uses `any()` instead of `all()`
File: `whitebox/code/board.py`
Error: Owning one property in a color group incorrectly counted as monopoly ownership.
Fix: Changed `any()` to `all()` in the ownership check.

Bug 8: Trade Does Not Credit Seller
File: `whitebox/code/game.py`
Error: Trade deducted buyer cash and transferred ownership but did not credit seller cash.
Fix: Added seller cash credit equal to trade amount.

Bug 9: Bank Collects Negative Amounts
File: `whitebox/code/bank.py`
Error: `Bank.collect()` allowed negative values, reducing bank funds incorrectly.
Fix: Added a `ValueError` for negative collection amounts.

Bug 10: Loan Disbursement Does Not Deduct Bank Funds
File: `whitebox/code/bank.py`
Error: Loan amount was given to player but not subtracted from bank total funds.
Fix: Deducted the loan amount from `self.total_funds`.

5.4 Errors or logical issues found by white-box reasoning

In this phase, the above ten bugs are the complete error set documented under "All Bugs and Fixes (Phase 1.3)".

5.5 Required error-fix commit format

For each confirmed logic defect, save with required format:

- `Error 1: Fix incorrect dice range to 1-6`
- `Error 2: Include properties in net worth calculation`
- `Error 3: Award GO salary when passing GO`
- `Error 4: Allow purchase when cash equals property price`
- `Error 5: Transfer rent to owner correctly`
- `Error 6: Use max net worth for winner selection`
- `Error 7: Use all() for monopoly check`
- `Error 8: Credit seller during trade`
- `Error 9: Block negative bank collect amounts`
- `Error 10: Deduct loan amount from bank funds`

6 Evidence Summary

- Existing tests already covered selected happy paths (purchase, rent, winner, basic trade).
- White-box analysis highlighted missing branch checks in jail and invalid-input trade paths.
- Pylint identified style and maintainability issues suitable for iterative cleanup commits.